



Clase 5. Mapping, QC and visualization

Sequence alignment

To quantify the degree of similarity between two sequences, first, we must align them and find the correct position in the reference genome from where the reads originate. But...

Repetitive regions: Short reads can map several times in different locations of the genome

Genetic Variants: The alignment must allow a certain degree of flexibility

Sequence alignment

To quantify the degree of similarity between two sequences, first, we must align them.

Substitution or Mismatch: change in one base for another

```
-CGATGCTAGCGTATCGTAGTCTATCGTAC
||| ||| ||| ||| ||| ||| ||| ||| |||
ACGATGCTAGCGTTTCGTA-TC-ATCGTA-
```

Gap: Is a deletion or an insertion

Depending on the allowed number of gaps, we might obtain different alignments

Without Gaps (10 Match)

```
a:  ATATTGCTACGTATATCAT
      ||| ||| ||| |||
b:  ATATATGCTACGTATCAT
```

With Gaps (14 Match)

```
a:  ATAT-TGCTACGTATATCAT
      ||| ||| ||| ||| |||
b:  ATATATGCTACGTATCAT
```

With Gaps in a and b (16 Match)

```
a:  ATAT-TGCTACGTATATCAT
      ||| ||| ||| ||| |||
b:  ATATATGCTACG--TATCAT
```

Two types of alignment

Global Alignment

Target Sequence

```
5' ACTACTAGATTACTTACGGATCAGGTACTTTAGAGGCTTGCAACCA 3'
   ||||| ||||| ||||| |||||
5' ACTACTAGATT---ACGGATC--GTACTTTAGAGGCTAGCAACCA 3'
```

Query Sequence

- Performed **end-to-end** for both sequences **introducing gaps** if needed
- Aligns all the bases both for query and target
- Ideal for related sequences with similar length
- Uses the **NEEDLEMAN-WUNSCH** algorithms to **minimize** the number of mismatches, deletions, and insertions

Local Alignment

Target Sequence

```
5' ACTACTAGATTACTTACGGATCAGGTACTTTAGAGGCTTGCAACCA 3'
      ||| ||||| ||||| |||||
```

Query Sequence

```
5' TACTCACGGATGAGGTACTTTAGAGGC 3'
```

- Perform alignments only in the most similar regions
- Align pieces of query and target sequences (substrings/subsequences)
- Can provide more than one alignment
- Uses the **SMITH-WATERMAN** Algorithms

Alignment Score

To compare different alignments, we score them... scoring will consider:

Number of matches, Identity and similarity, Number of Gaps

The **Scoring Scheme** is a set of rules which assign the alignment score to any given alignment of two sequences.

- It is residue-based: consists of **residue substitution score** (score for each possible residue alignment), plus penalties for gaps (initiation and extension)
- Alignment score is the sum of substitution scores and gap penalties

Example: Let's give a +1 for match and -1 for substitutions and ignore gaps

The best alignment would be:

ATGGCGT	
ATG-AGT	score: $+1 + 1 + 1 + 0 - 1 + 1 + 1 = 4$

Alternative alignment:

ATGGCGT	
A-TGAGT	score: $+1 + 0 - 1 + 1 - 1 + 1 + 1 = 2$

Different alignment programs will have different scoring matrix

Scoring Matrix

It is a 4x4 nucleotides matrix, accounting for purine/pyrimidine mutations

	C	T	A	G
C	2	1	-1	-1
T	1	2	-1	-1
A	-1	-1	2	1
G	-1	-1	1	2

Score Scheme:

Match +2

Mismatch purine/pyrimidine +1

Mismatch -1

Different alignments programs will have different substitution matrix and gaps penalties

RNA-sequencing mapping is challenging

- It requires high computational power (Millions of reads being analyzed)
- Intron presence (RNA libraries and mapping to the reference genome)
- Inefficient alignment (Genetic variants, repeat sequences and contaminations)

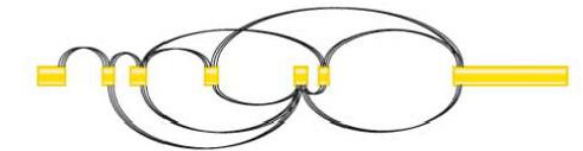
RNA-sequencing Alignment strategies

Currently, 3 different approaches exist:

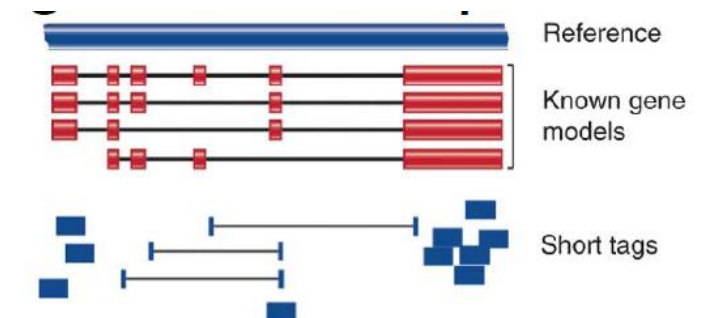
- *De novo Assembly* (No reference genome and reads are aligned to assemble transcripts)
- *Genome alignment + Gene model assembly* (Reference genome available but gene models are unknown or bad quality)
- *Transcriptome alignment* (Available genome and gene models)



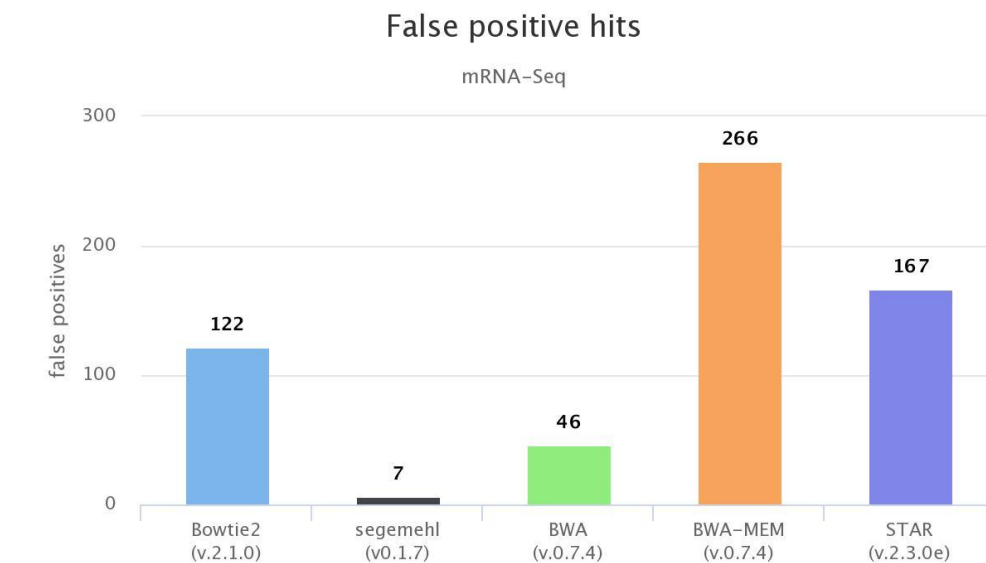
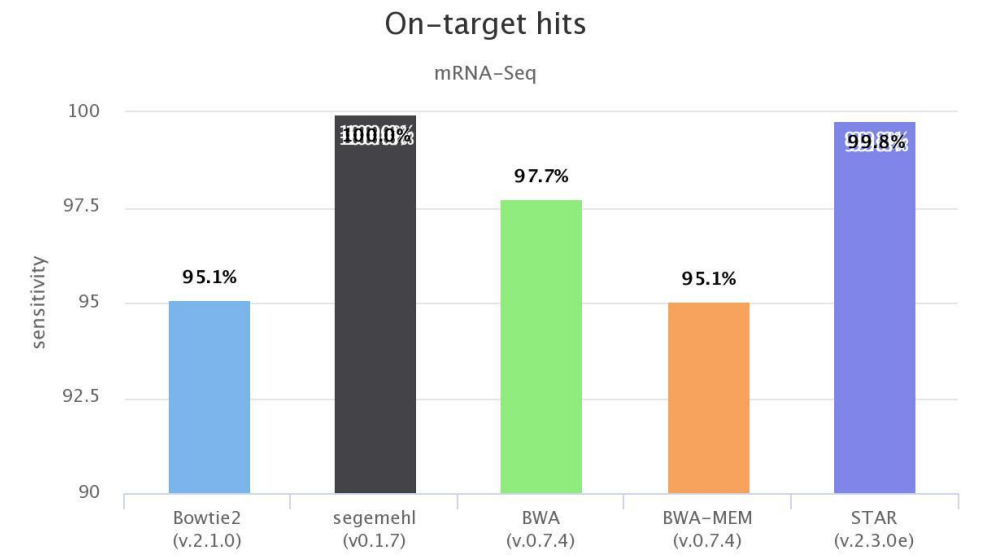
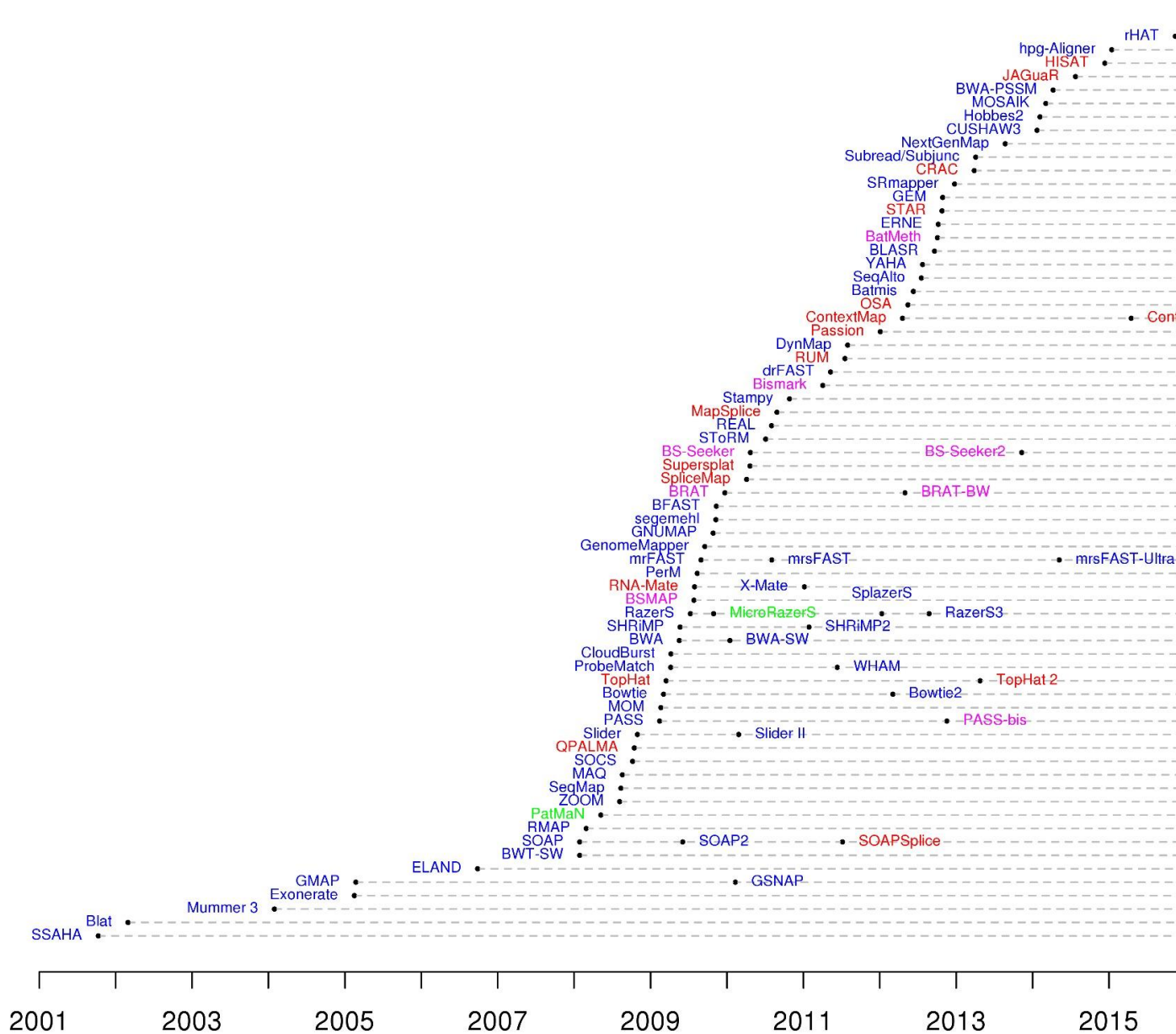
Assemble transcripts from overlapping tags



Infer possible transcripts and abundance

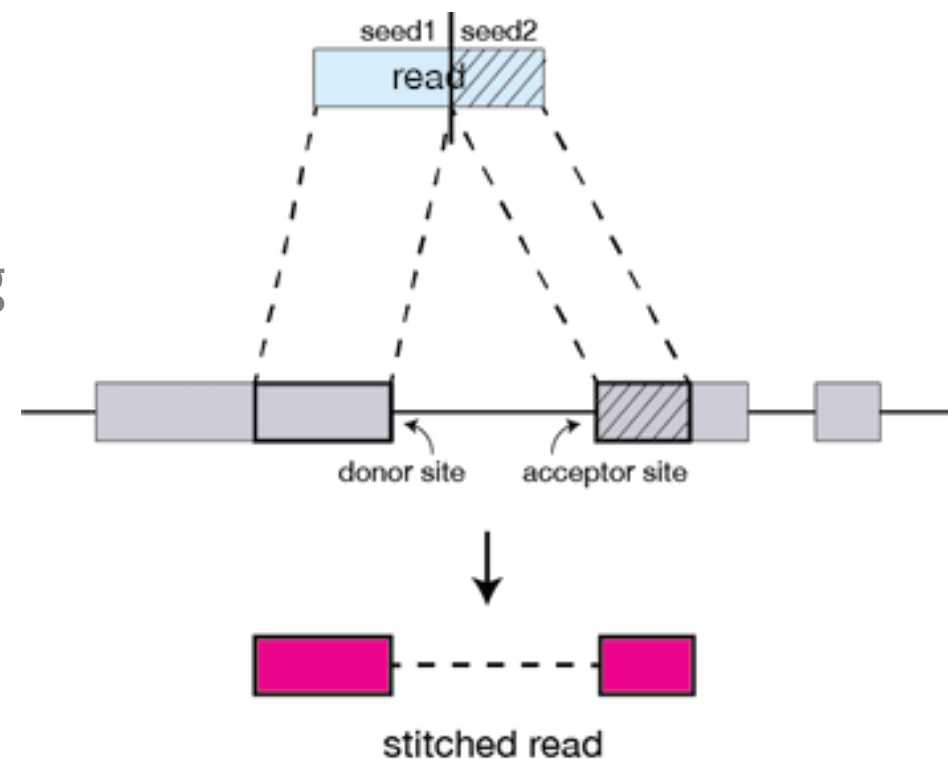


Available reads aligners



Spliced alignment in RNA-seq data analysis

- Sequenced fragments come from **mature transcripts** (without introns)
- Alignments are performed using as a base the reference genome (including intronic sequences)
- Short reads can span big introns in the reference genome (big gaps)
- Spliced alignment will require reads from 35-50bp minimum



Features of alignment strategies

- Genome alignment + Gene model assembly

Positive:

Detection of new transcripts

Negative:

Alignment is difficult

Insert size and inner distance are difficult to infer due to the intron presence

- Transcriptome alignment

Positive:

Do not require spliced alignment

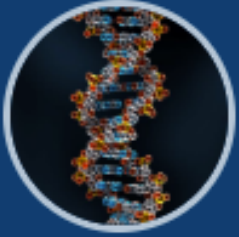
Simplifies the expression estimation by isoform

Insert size and inner distance are informative

Negative:

Depends on the gene model quality

No discovery of new transcripts 😞



HISAT2

graph-based alignment of next generation sequencing reads to a population of genomes

- Based on a modified Burrows-Wheeler transform (Data transformation to compress and store reference sequence. This transformation makes the alignment faster. But the improvement of this program is that it even compresses the reference using a Hierarchical Graph FM index (HGFM)
- Implements a graph-based data structure and an alignment algorithm to enable fast and sensitive alignment of sequencing reads to a genome and a large collection of small variants.
- Implements an indexing algorithm for repeat sequences
- HISAT-genotype uses HISAT2 as an alignment engine along with additional algorithms to carry out HLA typing and DNA fingerprinting analysis

OUTPUT

Mapped Reads (BAM files)

Summary report

Principle of Burrows-Wheeler data transformation

Uses a **Hash/kmer table** of the reference/query sequence, finds **kmer exact matches**, looks to the reference genome, and **score** alignment.

Little reference Genome: **CATGGTCATTGGTTCC**

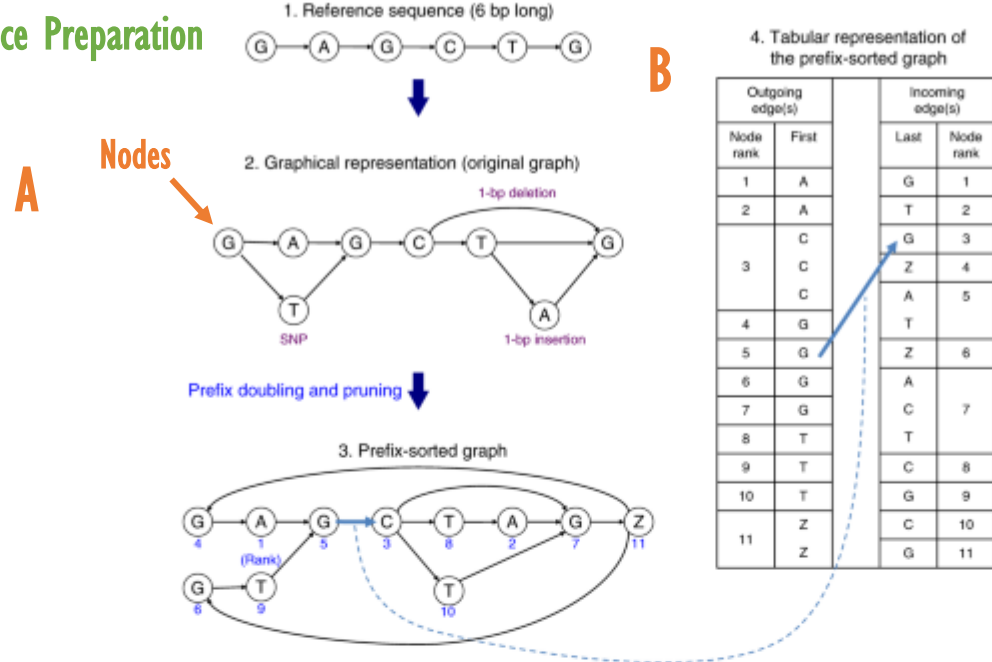
k=3	K-mer/Hash	Positions
	CAT	1,7
	ATG	2
	TGG	3,10
	GGT	4,11
	GTC	5
	TCA	6
	ATT	8
	TTG	9
	GTT	12
	TTC	13
	TCC	14

Read: **TGGTCA**

Hash/kmer table can be used to quickly find candidate alignment locations in the reference.

The Kmer TGG is assigned to position 3 and 10, while Kmer TCA is assigned to 6

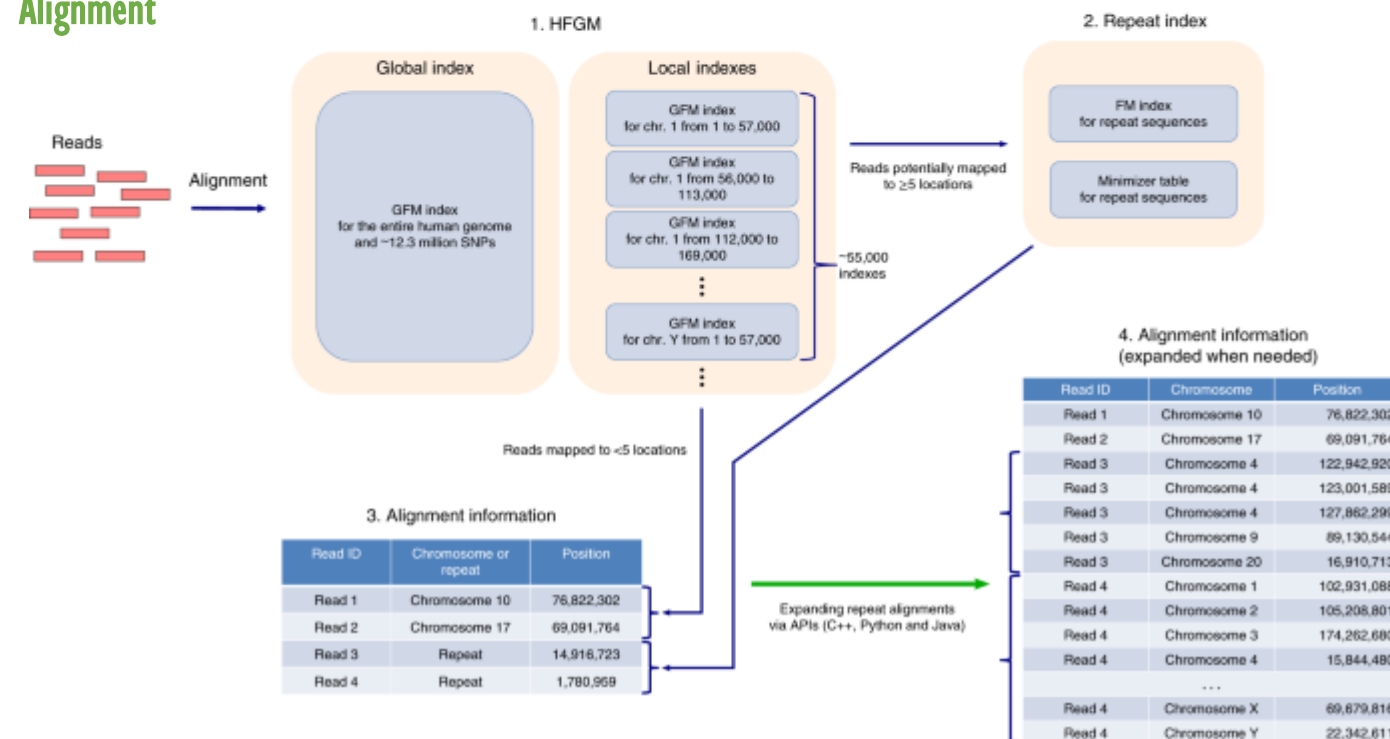
Reference Preparation



- A. Linear graph of the reference genome and then adds mutations deletions and insertions as alternative paths through the graph.
 → Every path describes a sequence that can occur in the reference genome or its variants

- B. The graph is stored in a table that is equivalent (describing nodes Out and incoming)

Alignment



- Additionally, hierarchical GFM (Global and local) are created.
 → Local regions are better for RNA-seq reads spanning multiple exons
 → When the search is local is always faster

Alignment output (BAM/SAM)

```
@HD VN:1.6 SO:coordinate
```

```
@SQ SN:ref LN:45
```

```
r001 99 ref 7 30 8M2I4M1D3M = 37 39 TTAGATAAAGGATACTG *
```

```
r002 0 ref 9 30 3S6M1P1I4M * 0 0 AAAAGATAAGGATA *
```

```
r003 0 ref 9 30 5S6M * 0 0 GCCTAAGCTAA *
```

```
r004 0 ref 16 30 6M14N5M * 0 0 ATAGCTTCAGC *
```

```
r003 2064 ref 29 17 6H5M * 0 0 TAGGC *
```

```
r001 147 ref 37 30 9M = 7 -39 CAGCGGCAT *
```

Opcional

```
SA:Z:ref,29,-,6H5M,17,0;
```

```
SA:Z:ref,9,+,5S6M,30,1;
```

```
NM:i:1
```

Col	Field	Type	Regexp/Range	Brief description
1	QNAME	String	[!-?A-~]{1,254}	Query template NAME
2	FLAG	Int	[0, 2 ¹⁶ - 1]	bitwise FLAG
3	RNAME	String	* [!-()+-<>-~] [!-~]*	Reference sequence NAME
4	POS	Int	[0, 2 ³¹ - 1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0, 2 ⁸ - 1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [!-()+-<>-~] [!-~]*	Ref. name of the mate/next read
8	PNEXT	Int	[0, 2 ³¹ - 1]	Position of the mate/next read
9	TLEN	Int	[-2 ³¹ + 1, 2 ³¹ - 1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.] +	segment SEQUENCE
11	QUAL	String	[!-~] +	ASCII of Phred-scaled base QUALity+33

Second field: FLAGS describing alignment

```
@HD VN:1.6 SO:coordinate
@SQ SN:ref LN:45
r001 99 ref 7 30 8M2I4M1D3M = 37 39 TTAGATAAAGGATACTG *
r002 0 ref 9 30 3S6M1P1I4M * 0 0 AAAAGATAAGGATA *
r003 0 ref 9 30 5S6M * 0 0 GCCTAAGCTAA * SA:Z:ref,29,-,6H5M,17,0;
r004 0 ref 16 30 6M14N5M * 0 0 ATAGCTTCAGC *
r003 2064 ref 29 17 6H5M * 0 0 TAGGC * SA:Z:ref,9,+,5S6M,30,1;
r001 147 ref 37 30 9M = 7 -39 CAGCGGCAT * NM:i:1
```

Col	Field	Type	Regex/Range	Brief description
1	QNAME	String	[!-?A-~]{1,254}	Query template NAME
2	FLAG	Int	[0, 2 ¹⁶ - 1]	bitwise FLAG
3	RNAME	String	* [!-()+-<>-~] [!-~]*	Reference sequence NAME
4	POS	Int	[0, 2 ³¹ - 1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0, 2 ⁸ - 1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [!-()+-<>-~] [!-~]*	Ref. name of the mate/next read
8	PNEXT	Int	[0, 2 ³¹ - 1]	Position of the mate/next read
9	TLEN	Int	[-2 ³¹ + 1, 2 ³¹ - 1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEquence
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

Explaining Alignment through a 12bit FLAG

Bit	Description
1	0x1 template having multiple segments in sequencing
2	0x2 each segment properly aligned according to the aligner
4	0x4 segment unmapped
8	0x8 next segment in the template unmapped
16	0x10 SEQ being reverse complemented
32	0x20 SEQ of the next segment in the template being reverse complemented
64	0x40 the first segment in the template
128	0x80 the last segment in the template
256	0x100 secondary alignment
512	0x200 not passing filters, such as platform/vendor quality controls
1024	0x400 PCR or optical duplicate
2048	0x800 supplementary alignment

Bits are combined, by seeming them, to describe alignment <https://broadinstitute.github.io/picard/explain-flags.html>

Sixth field: CIGAR reconstruct alignment

Alignment

```
Coord 12345678901234 5678901234567890123456789012345
ref    AGCATGTTAGATAA**GATAGCTGTGCTAGTAGGCAGTCAGCGCCAT

+r001/1      TTAGATAAAAGGATA*CTG
+r002        aaaAGATAA*GGATA
+r003        gcctaAGCTAA
+r004                ATAGCT.....TCAGC
-r003                ttagctTAGGC
-r001/2                                CAGCGGCAT
```

```
@HD VN:1.6 SO:coordinate
@SQ SN:ref LN:45
r001 99 ref 7 30 8M2I4M1D3M = 37 39 TTAGATAAAAGGATACTG *
r002 0 ref 9 30 3S6M1P1I4M * 0 0 AAAAGATAAGGATA *
r003 0 ref 9 30 5S6M * 0 0 GCCTAAGCTAA * SA:Z:ref,29,-,6H5M,17,0;
r004 0 ref 16 30 6M14N5M * 0 0 ATAGCTTCAGC *
r003 2064 ref 29 17 6H5M * 0 0 TAGGC * SA:Z:ref,9,+,5S6M,30,1;
r001 147 ref 37 30 9M = 7 -39 CAGCGGCAT * NM:i:1
```

- Sequences of numbers and bases associated with an operation
- Describing per base the alignment to the reference

Op	BAM	Description	Consumes query	Consumes reference
M	0	alignment match (can be a sequence match or mismatch)	yes	yes
I	1	insertion to the reference	yes	no
D	2	deletion from the reference	no	yes
N	3	skipped region from the reference	no	yes
S	4	soft clipping (clipped sequences present in SEQ)	yes	no
H	5	hard clipping (clipped sequences NOT present in SEQ)	no	no
P	6	padding (silent deletion from padded reference)	no	no
=	7	sequence match	yes	yes
X	8	sequence mismatch	yes	yes

BAM file manipulation programs

- Several bioinformatics tools are commonly used to manipulate BAM files

SamTools

Bamtools

Picard

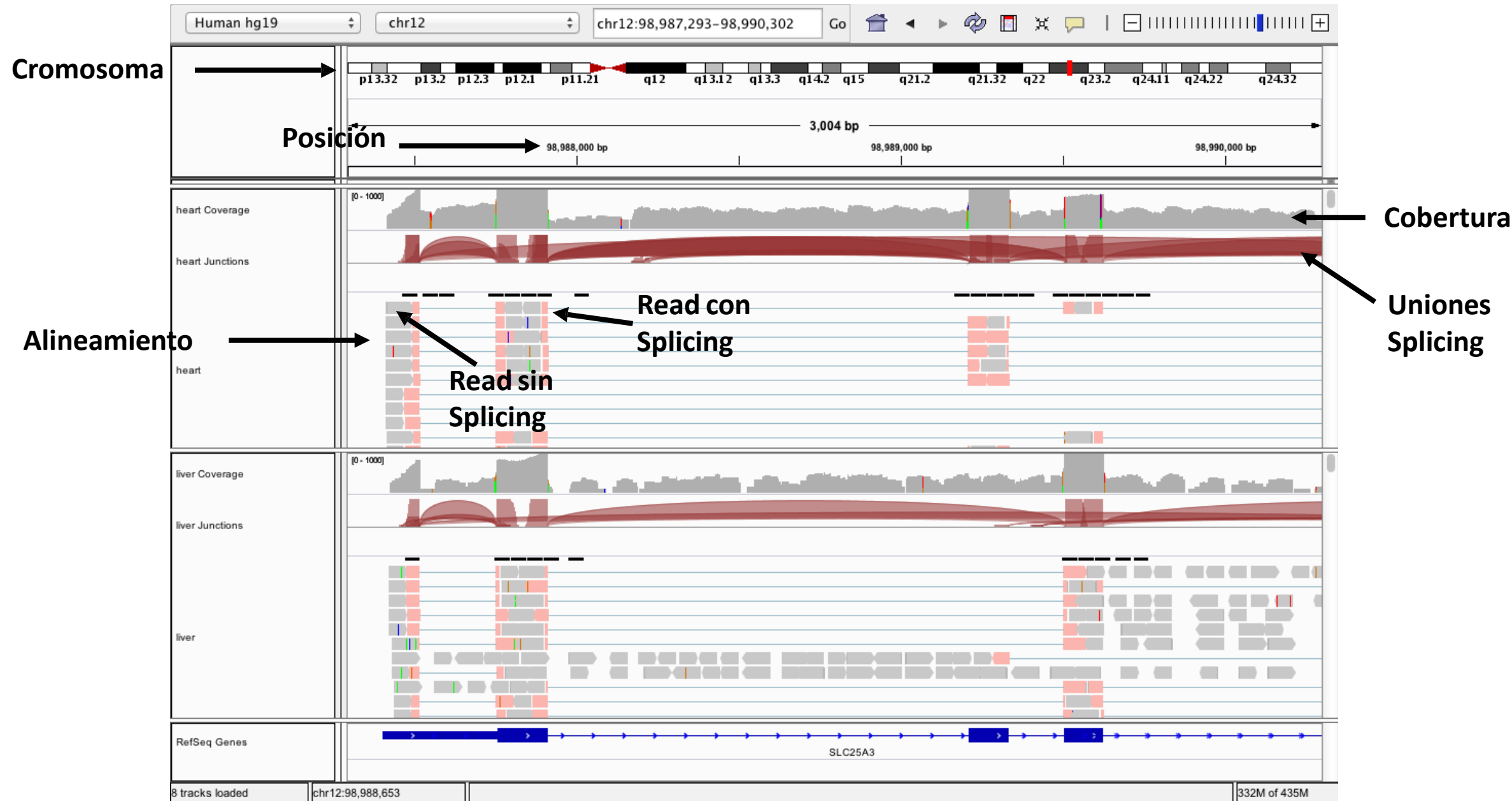
Note: Many further analyses will need the BAM files organized by alignment coordinate or name of the read

- For BED files

Bedtools

Bedops

Alignment visualization with IGV



Alignment QC

Once the alignment is done, it is important to perform a mapping quality check:

Alignment Statistics:

Number of mapped reads
Percentage of unmapped reads



Provided by the aligner

It will depend on the quality of your reference genome

Zebrafish >80% mapped reads

Mouse >90% mapped reads

Human >95% mapped reads

3' and 5' Bias
Junctions saturation
PCR duplicates
Inner size



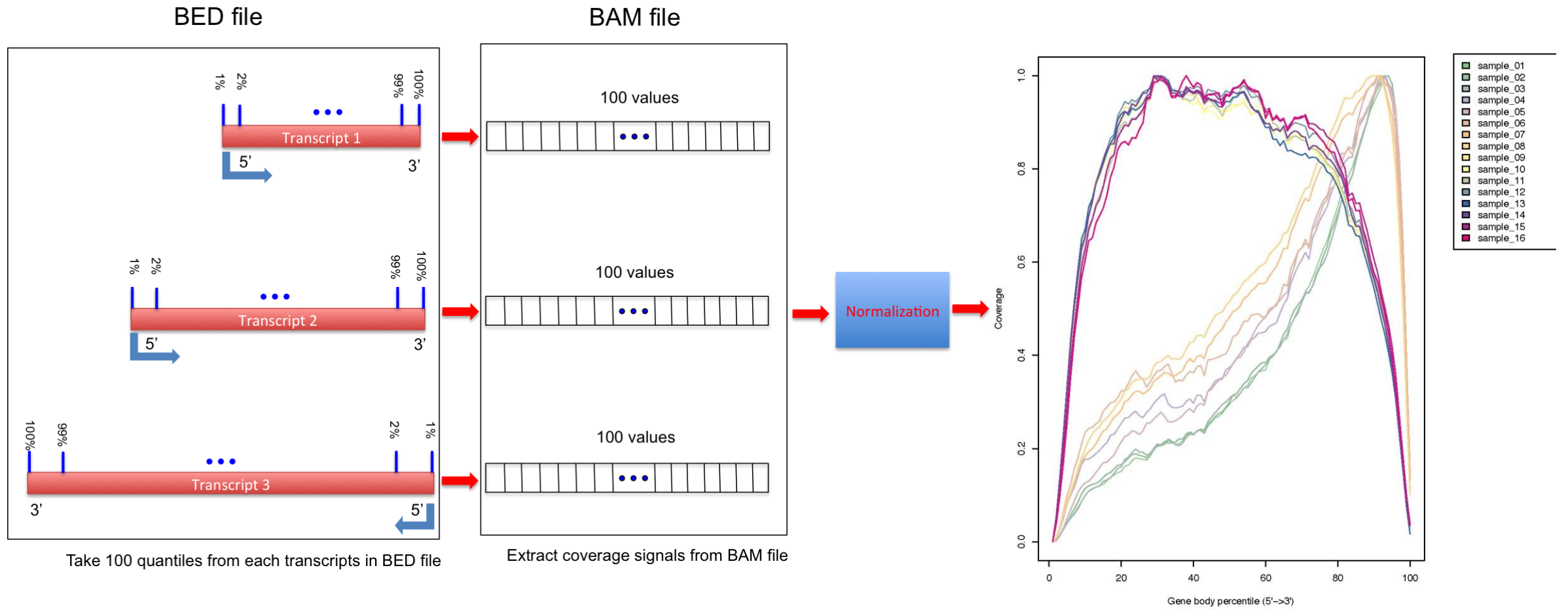
RSeQC



<http://rseqc.sourceforge.net/>

RseQ: 3' and 5' Bias

geneBody_coverage.py



<http://rseqc.sourceforge.net/#genebody-coverage-py>

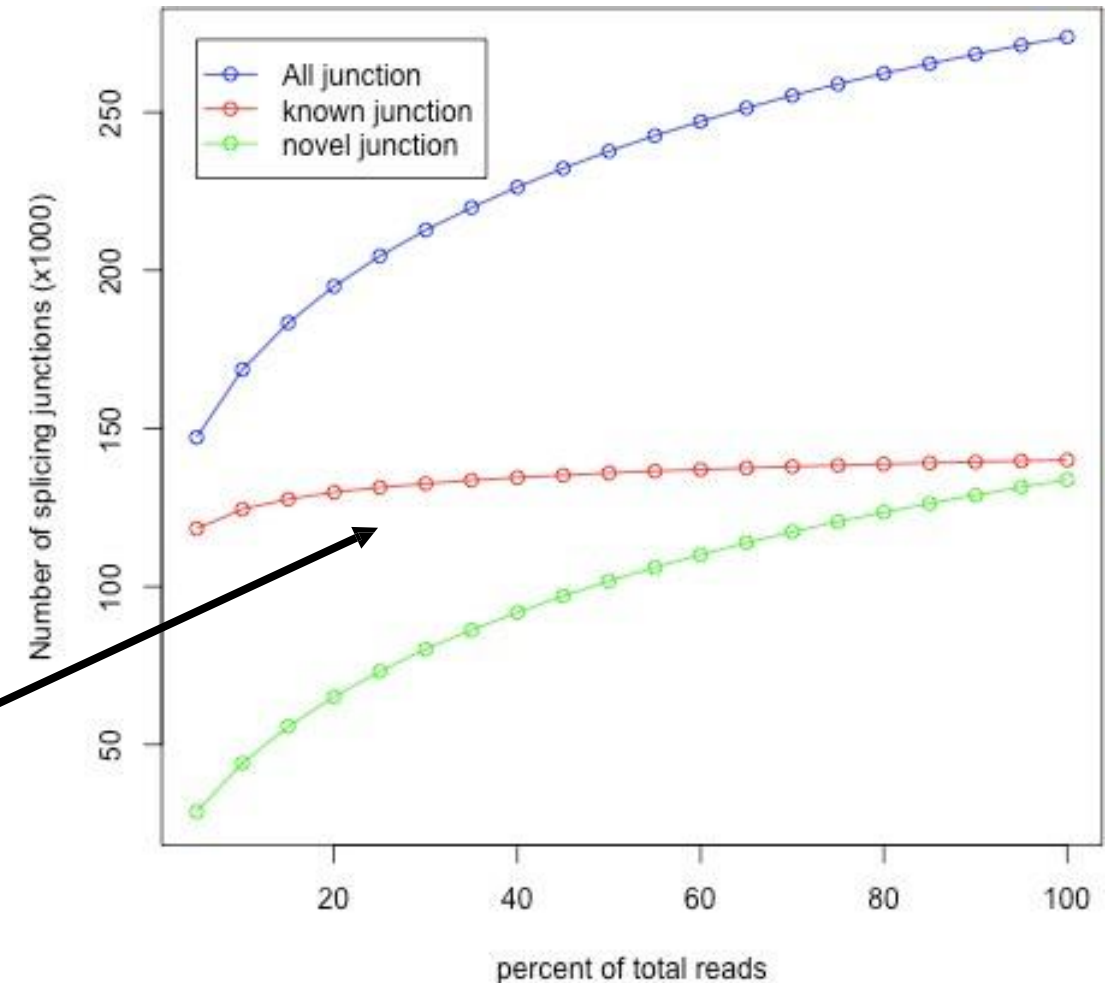
RseQ: Saturación de uniones splicing

Junction_saturation.py

It verifies if sequencing coverage is enough to perform splicing analysis

If the transcriptome is saturated, then no new junction must appear with the subsampling

Known junctions are saturated



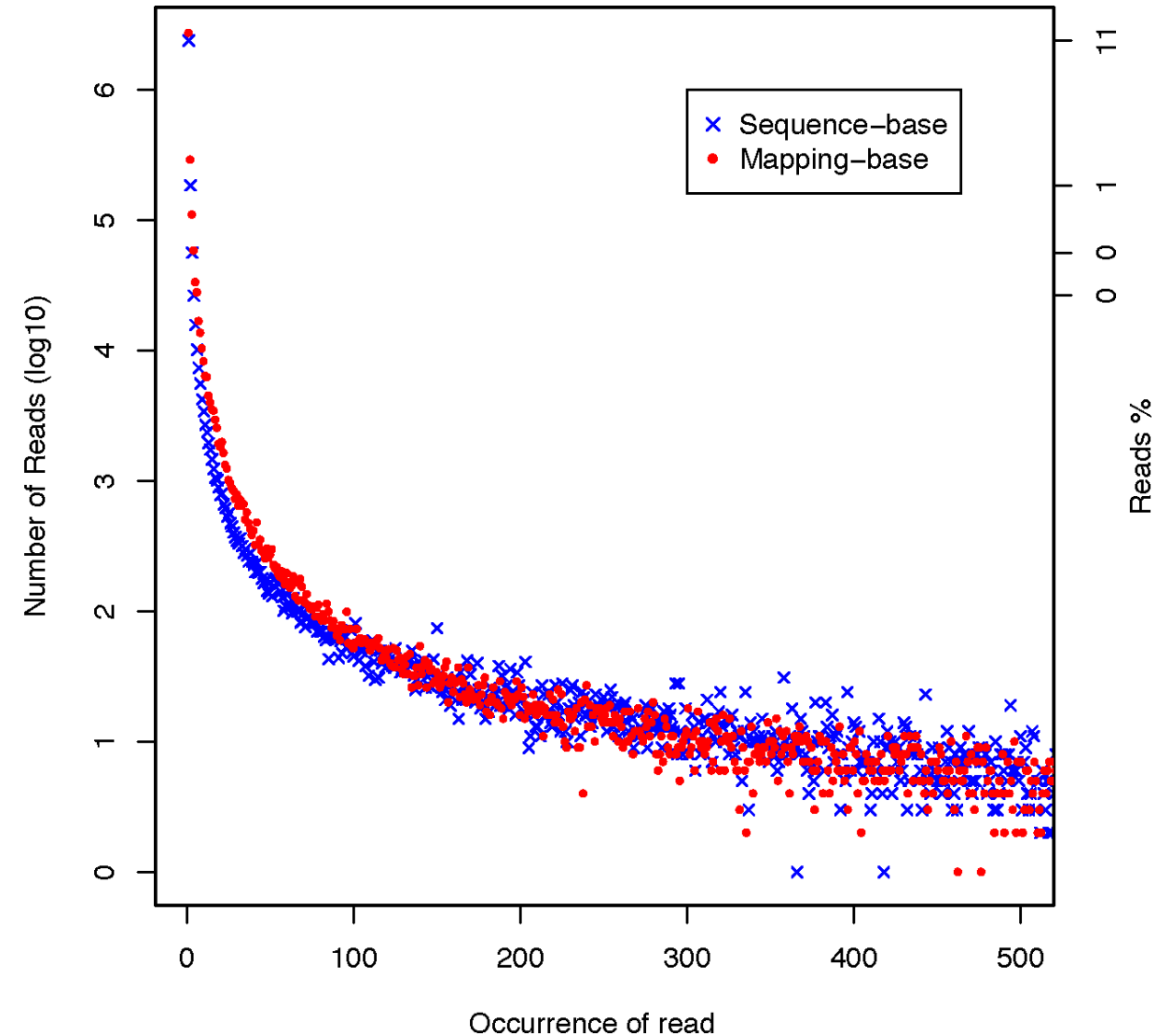
RseQ: duplicados de PCR o PCR bias

read_duplication.py

Sequenced based: Identical read sequences

Mapping based: Genomic positions

Spliced reads: Must start at the same position and display the same splicing pattern.



<http://rseqc.sourceforge.net/#read-duplication-py>

Let's move to the practical section II

But first...Coffee!

